IT UNIVERSITY OF COPENHAGEN

Final Report

Master of Science in Computer Science
IT-University of Copenhagen

Course Name: DevOps: Software Evolution and Software Maintenance
Course Code: KSDSESM1KU
Submission Date: 17/05/2026

Author	**Email**
Jacob Sonne	json@itu.dk
Renate Mekere	rmek@itu.dk
Adam Nørgård Aabye	aaab@itu.dk
Marc David Paget	mdpa@itu.dk
Sebastian Thiel Steensgaard	sebs@itu.dk
Asger Engelund Trads	asgt@itu.dk

Table of contents

1 System's Perspective 1
1.1 Design and Architecture 1
1.2 Dependencies 4
1.3 Current System State 6
2 Process' perspective 6
2.1 CI/CD pipelines 6
2.2 Monitoring 8
2.3 Logging 9
2.4 Security Hardening 10
2.5 Availability and scaling 12
3 Reflection Perspective 13
3.1 DevOps Style of Work 13
3.2 Key Challenges and Lessons Learned 14
3.3 Evolution and Refactoring 14
3.4 Operation 15
3.5 Maintenance 15
4 Use of Generative AI 16
5 Appendix 17
5.1 Monitoring dashboards 17
5.2 Logging dashboard 18

1 System's Perspective

1.1 Design and Architecture

The Minitwit system is deployed on DigitalOcean using three droplets: one PostgreSQL database node, one Docker Swarm manager node, and one Docker Swarm worker node. The application is containerized and deployed through Docker Swarm, which runs and manages the application services across the swarm.

The team have implemented a PowerShell-based deployment script, `scripts/deploy-existing.ps1`. The script deploys to already existing droplets by connecting to the server with SSH, copying configuration files with SCP, setting the required environment variables, and deploying the services using Docker Swarm.

The public entry point to the system is Nginx. Incoming traffic is routed through Nginx, which forwards requests to the web service, the API service, or Grafana based on the URL path. The API and web services communicate with the PostgreSQL database for persistent storage. The simplified request flow is:

`Browser / Simulator` → `Nginx` → `Web / API` → `PostgreSQL`

The system also includes a full monitoring stack comprising Prometheus, Grafana, Loki, Promtail, cAdvisor, and node-exporter, which is described in detail in Section 2.2.

The deployment architecture is illustrated in Figure 1.

A `scripts/create-infrastructure.ps1` script was also implemented to automate full infrastructure creation from scratch. It creates the three droplets, installs PostgreSQL and Docker, configures Docker Swarm, joins the worker node, and restores the database from a backup. A companion script `scripts/backup-database.ps1` backs up the live PostgreSQL database before any redeployment. Testing of the full infrastructure creation was limited because the DigitalOcean account reached the three-droplet limit, preventing creation of new droplets while the existing ones were running.

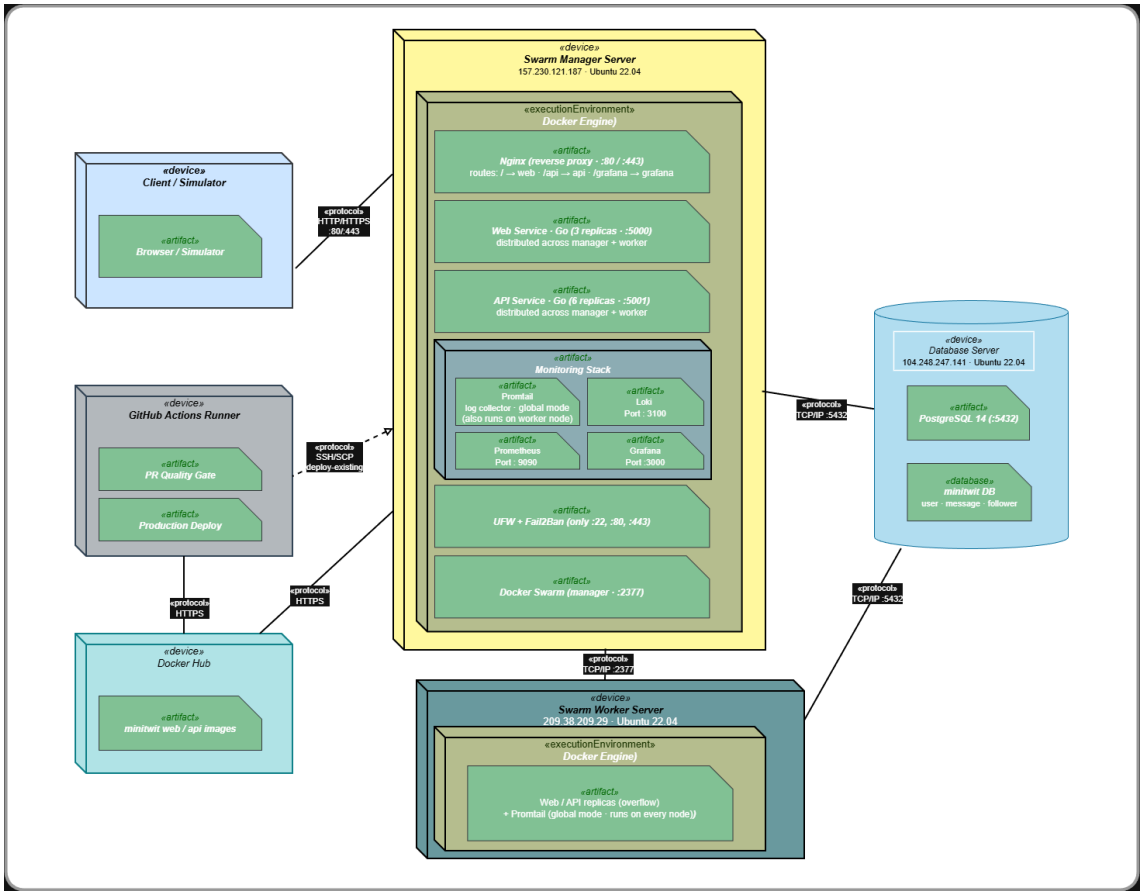


Figure 1: Deployment diagram of the MiniTwit system

Additionally two sequence diagrams were created to visualize the sequential flow of logging in and posting a message in the solution. The interaction can be seen in figure 2 and 3.

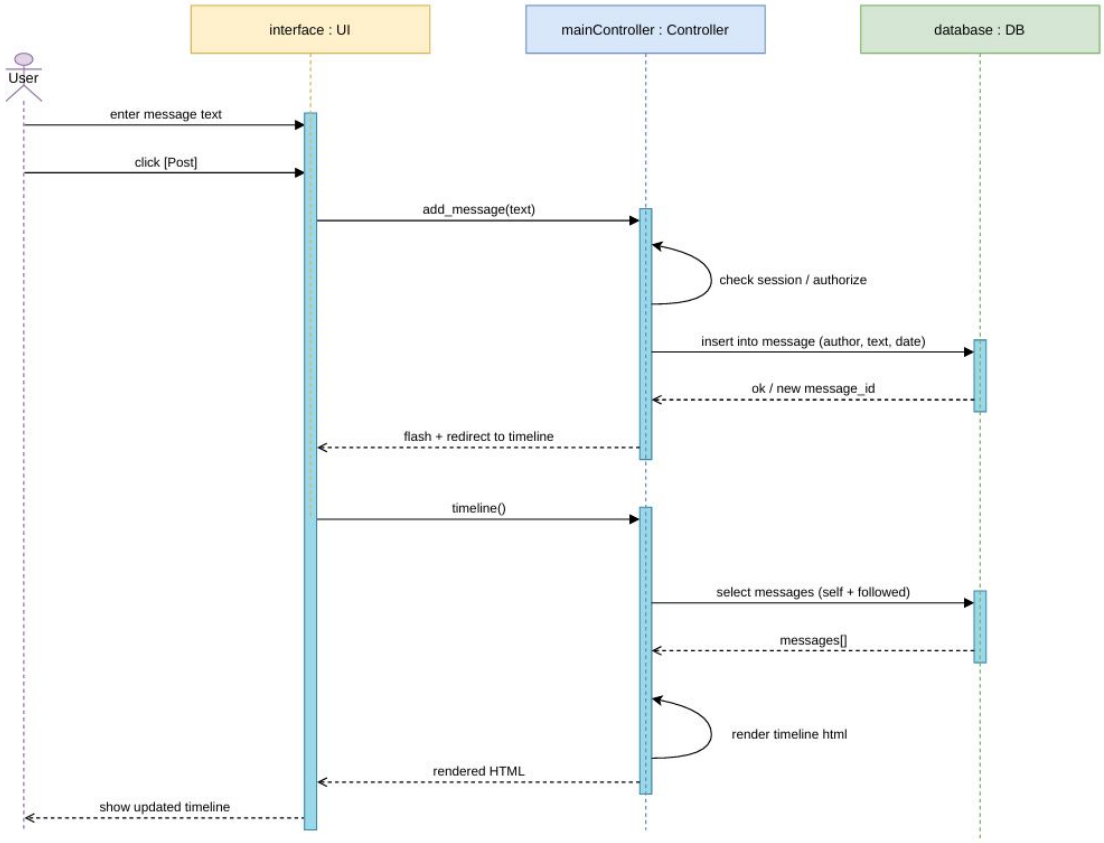


Figure 2: Post creation and timeline rendering sequence diagram

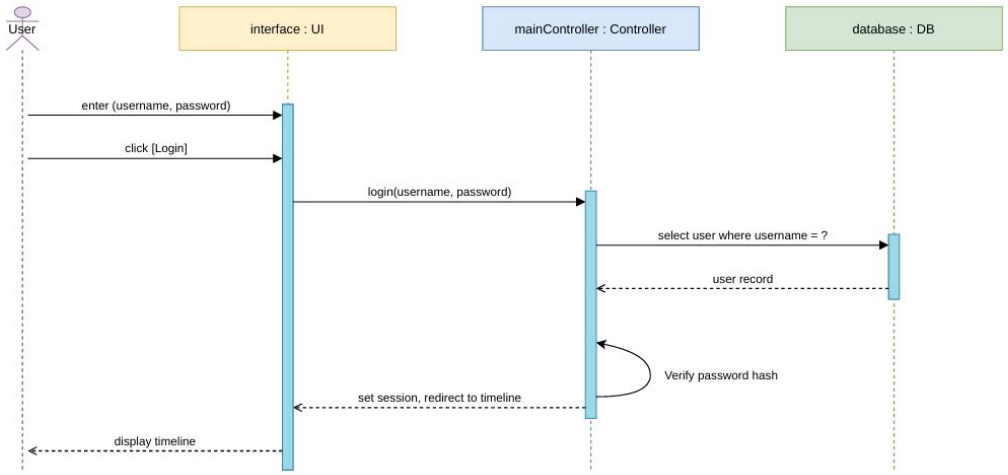


Figure 3: Login sequence diagram of the MiniTwit system

1.2 Dependencies

The following are the dependencies for our Minitwit system, listed in an organized manner in order to better provide an overview.

Runtime Dependencies

These dependencies include the environment to run the application on including the operating system, VM technologies and programming language.

- Ubuntu Linux: Operating System
- Docker (swarm mode): Virtual Machine
- golang:1.25 (Docker image)

Application Libraries

This list contains the libraries and what each library is for.

- Go 1.25.6: Programming language.
- Gorilla Mux: HTTP request routing.
- Gorilla Sessions: Session management.
- GORM: Object-relational mapper.
- PostgreSQL Driver: Database connectivity.
- Zerolog: Structured logging.
- Prometheus client_golang: Metrics instrumentation.

Testing

- Python 3.11: Programming language
- pytest: Testing framework

DevOps / Build Dependencies

This list includes tools for employing DevOps practices.

- Github: Code repository
- Dockerhub: Docker image repository
- Github Actions: CI/CD Pipeline
- Codacy: Quality assurance
- SonarQube: Quality assurance
- Trivy: Pipeline hardening
- Fail2Ban: Intrusion-prevention framework

1.3 Current System State

The application has certain issues, which have not been resolved due to lack of time. These issues are mentioned below.

- Ideally we would have more nodes, and more managers to ensure that even if a manager dies, another manager can still remain in control of the swarm.
- There is a bug where it is possible to create multiple instances of the same user.
- There are currently no logging for the WEB part of the application. This has been neglected as almost all traffic has come from the simulator, which only talks with the API.
- Our entire monitoring stack as well as nginx don't have a health check configured, meaning we cannot monitor the uptime of these services
- We have used Trivy for analysing dependencies in in the docker images, but have not acted on the information.

2 Process' perspective

2.1 CI/CD pipelines

Our project mainly use two separate pipelines called "PR Quality Gate" and "CI/CD Pipeline Production Server". Both pipelines utilizes a pipeline template called "Merged Test Suite", which will be referred to as the "test-suite".

PR Quality Gate pipeline

This pipeline functions as a quality gate whenever we create a Pull Request (PR) into our development branch called "develop". Each time a PR is created or updated, the pipeline would run some static analysis checks, build the solution, and test if our test-suite passes. An overview of the pipeline can be seen in figure 4. Additionally SonarQube and Codacy has been set up for the git repository such that every time a Pull Request is created it will scan the code.

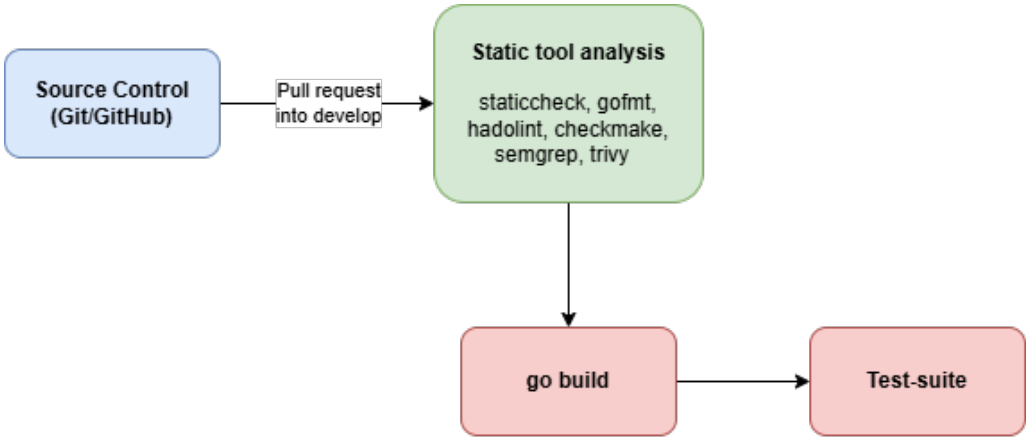


Figure 4: Overview of the PR Quality gate pipeline

CI/CD Pipeline Production Server pipeline

This pipeline also runs the quality checks as the ”PR Quality Gate” pipeline, however after that it also pushes our docker images to a docker registry and deploys our application in our production environment. The pipeline is triggered each time an update is triggered on the main branch. It only deploys if every prior step succeeds.An overview of this pipeline can be seen in figure 5.

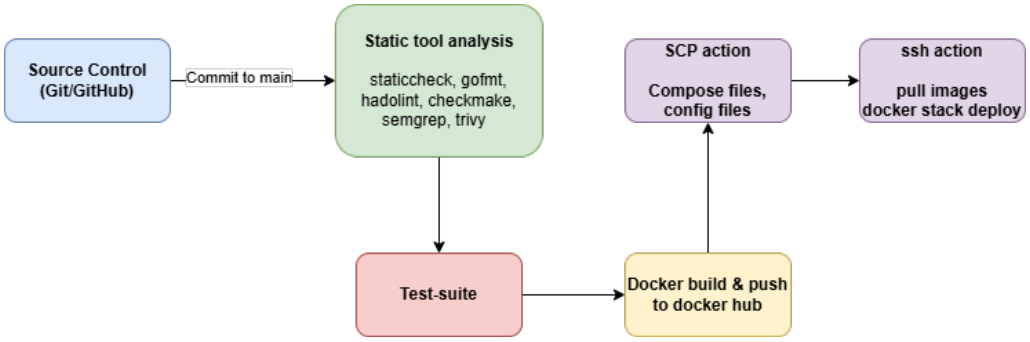


Figure 5: Overview of the CI/CD pipeline

Test-suite

The test suite is a template pipeline that is responsible for testing that the applications passes UI tests, unit tests and API tests. An overview of the pipeline template

can be seen in figure 6.

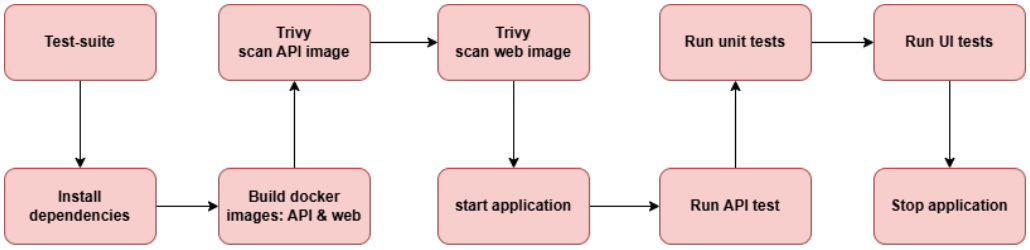


Figure 6: Illustration of the test-suite's steps and tools

2.2 Monitoring

For monitoring, we use a combination of Prometheus, cAdvisor, Node Exporter, and Grafana. Prometheus acts as the central metrics collection and storage system and it periodically scrapes metrics from the configured exporters and make them available as a data source for Grafana. Node Exporter exposes host-level metrics from the nodes, such as CPU, memory, disk, and network usage, while cAdvisor exposes container-level metrics, such as per-service CPU core utilization broken down by Docker Swarm service name.

Grafana is used as the data visualization layer and we have configured two dashboards which can be seen in the appendix (section 5.1):

- **Availability Dashboard**: Tracks the uptime of the Minitwit API, Minitwit Web, cAdvisor, and Node Exporter using Prometheus' `up` metric. It shows each service's availability percentage over time, as well as a state timeline indicating when services were up or down. The dashboard can be seen in the appendix or here .
- **Resource Usage Dashboard**: Provides a detailed view of the infrastructure health, including CPU utilization (min, mean, max, and p95), the memory utilization (total, used, and available), disk utilization, and network throughput (bytes received and transmitted) .

In addition to the dashboards, Grafana is configured with alert rules based on the `up` metric for both the Minitwit API and the Minitwit Web services. If either service becomes unavailable, an alert is triggered, and a notification is sent to the team via a Discord webhook, allowing us to quickly respond to unexpected downtime.

2.3 Logging

We use a Go library called "zerolog" for structured logging. We chose zerolog because it supports multiple log levels, is lightweight and performant, and is widely used in the Go community.

To collect the logs from the Docker containers running in the swarm, we use Promtail. Promtail adds metadata labels to each log entry before forwarding them, making it easier to search and filter logs later. In our case, Promtail adds the following labels:

- `app`: Whether the log originates from the web or API service.
- `endpoint`: The logical endpoint that was called, e.g. `register`, `msg`, or `fllws_username`.
- `method`: The HTTP method of the request, e.g. `GET` or `POST`.
- `statusCode`: The HTTP response code returned, e.g. `200` or `404`.
- `level`: The log level of the entry, e.g. `info` or `error`.

The logs are then forwarded to Loki, which is our log aggregation system. Loki indexes the labels created by Promtail, enabling fast and efficient querying without indexing the full log content.

Since Grafana was already used for the monitoring dashboards, we also use it to visualize the aggregated logs through an **API Dashboard** (see appendix 5.2). The dashboard queries Loki and displays the following information:

- **Summary statistics**: Total requests, successful requests (info-level), errors, and overall error rate percentage across the selected time range.

- **Request time series**: A graph showing request volume per endpoint over time.
- **Per-endpoint breakdown**: A table showing total, successful and failed requests along with the error rate for each individual endpoint.
- **Raw log view**: A filterable view of raw log entries, supporting filtering by endpoint, HTTP method, and log level.

2.4 Security Hardening

A focused security assessment was conducted on the MiniTwit system to identify risks across infrastructure, CI/CD workflows, deployment configuration, and application behavior. The assessment was based on static analysis tools, manual inspection of workflows, firewall testing, and external port scanning.

The identified risks are summarized in Figure 7. The most critical risks were related to exposed network ports, insecure handling of secrets, and database instability under load.

Risk	Likelihood	Impact	Priority
Exposed network ports	Likely	High	Critical
Secrets in environment variables	Likely	Critical	Critical
Root containers	Likely	Critical	Critical
Database connection exhaustion	Likely	Critical	Critical
Mutable Docker image tags	Possible	High	High
Public monitoring exposure	Possible	High	High
SSH brute-force risk	Possible	High	High
Unpinned GitHub Actions	Possible	High	High
Inherited secrets in workflows	Possible	High	High

Figure 7: Overview of identified security risks categorized by likelihood and impact

Implemented Improvements

The most significant improvement was firewall hardening on all three nodes using UFW. A deny-by-default incoming policy was configured, and unnecessary public ports were removed. Previously exposed ports included Docker management ports (2375, 2376, 2377) and direct application ports (5000, 5001). After hardening, only essential services remained publicly accessible:

- SSH (22)
- HTTP (80)
- HTTPS (443)
- Docker Swarm management (2377)
- Docker Swarm node communication (7946)
- Docker overlay network (4789)

All application traffic is now routed through the Nginx reverse proxy, reducing the system's public attack surface. Firewall changes were validated using external port scanning, confirming that removed services were no longer reachable.

To mitigate brute-force attacks, Fail2Ban was installed and configured to block repeated failed SSH login attempts. Automatic system updates were also enabled to ensure that known vulnerabilities are patched regularly.

Security scanning was integrated into the CI/CD pipeline using tools such as Semgrep and Trivy as well as having SonarQube and Codacy enabled for the repository. These tools help detect insecure code patterns and misconfigurations before deployment.

HTTPS is enabled using Let's Encrypt certificates managed via Certbot with Nginx configured to terminate TLS on port 443.

Remaining Risks and Improvements

Despite these improvements, some risks remain:

- Secrets are stored in environment variables and should be moved to a secure secret management solution
- Prometheus and Promtail run with elevated privileges (root user) in order to access the Docker socket and container logs respectively
- Docker images use mutable tags, reducing deployment reproducibility
- Monitoring tools may be publicly accessible
- Database connection limits caused instability under load

Future improvements include restricting monitoring access, using Docker Secrets, running containers as non-root users, introducing database connection pooling, and improving backup strategies.

Overall, the implemented measures significantly improved the security posture of the system, while highlighting areas for further hardening.

2.5 Availability and scaling

To achieve high Availability we use Docker Swarm with an on-failure restart policy and a rolling update strategy. Docker Swarm allows us to have multiple replicas of our services. If one replica goes down, other replicas will still be up and running, meaning that we don't suffer downtime. The on-failure restart policy means that any replica which goes down, is rebooted automatically. The rolling update policy means that when we ship new features, the replicas are updated one at a time, which means we can achieve zero down time when making new releases.

Docker Swarm is also the answer to the question of scaling. By replicating our services we employ horizontal scaling, and with Swarm Mode also comes load balancing. This means that requests to our services are distributed among the nodes and containers to not overburden any single replica.

This once again means high availability since, if the worker dies the manager is still controlling the swarm and running containers, and if the manager dies, the worker node can still run existing containers though new deployments will be unavailable until the manager recovers The application infrastructure is described in detail in Section 1.1.

3 Reflection Perspective

3.1 DevOps Style of Work

Our development process combined feature development with continuous operation of a live system under load from the MiniTwit simulator. Compared to previous projects, this required us to consider deployment, monitoring, and system reliability as part of the development process, rather than as separate activities.

We organized work using GitHub Projects, where issues were tracked and progressed through stages such as *To Do*, *In Progress*, *Ready*, and *Done*. Changes were developed in small feature branches and integrated through pull requests into the `develop` branch. Automated CI checks and code reviews helped ensure that changes met quality standards before integration.

In addition to our technical workflow, collaboration and communication were central to how we applied DevOps principles in practice. We coordinated work through GitHub and communicated primarily via Discord, where we shared updates, issues, and progress. Regular meetings helped align the team, while informal collaboration, including pair programming, supported knowledge sharing and problem-solving.

We observed that some team members naturally took on more responsibility, highlighting the importance of distributing knowledge and responsibilities to avoid reliance on a few individuals.

3.2 Key Challenges and Lessons Learned

One of the main challenges was working with infrastructure and deployment. Initial plans, such as using Vagrant, had to be changed due to compatibility issues, which required us to adapt our approach and implement a simpler deployment solution based on scripting and Docker Swarm. Operating a live system also introduced challenges related to debugging and stability. When issues occurred, we relied on logs, monitoring tools, and team communication to identify and resolve problems quickly. This emphasized the importance of observability and fast feedback in maintaining system reliability.

Another key lesson was the value of automation. Moving toward an automated deployment process reduced errors, made the system easier to reproduce and made deployment faster and easier. At the same time, we learned that it is often necessary to balance ideal technical solutions with practical constraints such as time, tooling limitations, and team experience.

Finally, we observed that a strong team culture was essential. Open communication and a willingness to share problems made it easier to collaborate and improve the system continuously. Overall, our DevOps approach was iterative and adaptive, where both the system and the process evolved based on experience and feedback.

3.3 Evolution and Refactoring

Some of the challenges in the project was the transition from the original implementation to a refactored system. We as a group chose to work in Go. This decision was based on Go's great compatibility with container-based environments such as Docker, as well as its performance.

This however turned out to be a challenge since only one of the members in the team had prior experience with Go. This refactoring from Python to Go required a lot of adaptation to another type system with a different approach to error handling. This slowed down the refactoring process and its deployment a lot.

Another issue came during the separation of services. For this refactoring we initially attempted to split the whole system into a bunch of small individual services. For

the scale of the project, however and time constrains this approach quickly became too complex in between deployment and communication between services. Therefore we decided to make a simpler version that would focus on separating the web and the api services (see commit a8377da).

These issues showed a clear example of how important it is to be critical of implementations of systems. Attempting to introduce overly complex solutions too early can result in overengineering, especially in early-stage systems. Service boundaries should be introduced gradually rather than upfront.

3.4 Operation

Operating the system introduced challenges related to orchestration and deployment, some appeared when using Docker Swarm. Configuring the swarm setup and ensuring communication between nodes proved difficult, especially when dealing with networking and port configuration issues. For example, connecting worker nodes to the manager required either exposing ports (raising security concerns) or reconfiguring the swarm to use private IP addresses (see commit de9ece2).

There was also uncertainty about whether to combine Docker Swarm with Docker Compose or rely on a single orchestration approach. But sticking to Docker Swarm reduced its complexity and simplified networking decisions.

We discovered that our monitoring was not sufficient. The metrics and logs did not provide a full coverage of the health of our application. This made it difficult to detect and diagnose issues. Introducing more complete monitoring mechanisms later in the process improved system visibility but also revealed how critical it is to include observability early.

3.5 Maintenance

Maintaining the system was particularly challenging due to issues with the test suite and CI/CD pipeline. Tests were unstable for a significant part of the project, due to misconfigurations in the pipeline and inconsistencies in the test setup. This resulted in the test suite being only fully functional late in the project.

It took a lot to stabilize the testing environment, including restructuring container setups, fixing dependencies such as geckodriver, and ensuring proper database connectivity (see commit 355f28c). This included introducing health checks, improving logging on failure, and refining environment variable handling.

These issues made it difficult to confidently validate changes and slowed down development, particularly during refactoring phases. Once the test suite became stable and integrated into the CI/CD pipeline, it significantly improved deployments.

This showed how reliable testing infrastructure is critical for maintainability. Delaying the stabilization of tests leads to increased debugging time and reduced development speed, especially in a distributed, container-based system.

4 Use of Generative AI

AI tools such as ChatGPT and other LLMs have been utilized for assisting with code-developing and as sparring partner. LLMs have also been used for debugging, and to better understand how certain technologies work. For instance ChatGPT was used to pinpoint the differences in docker-compose files, depending on whether they were used in swarm mode. Additionally LLMs have been used to validate grammar throughout the report.

5 Appendix

5.1 Monitoring dashboards

Availability Dashboard

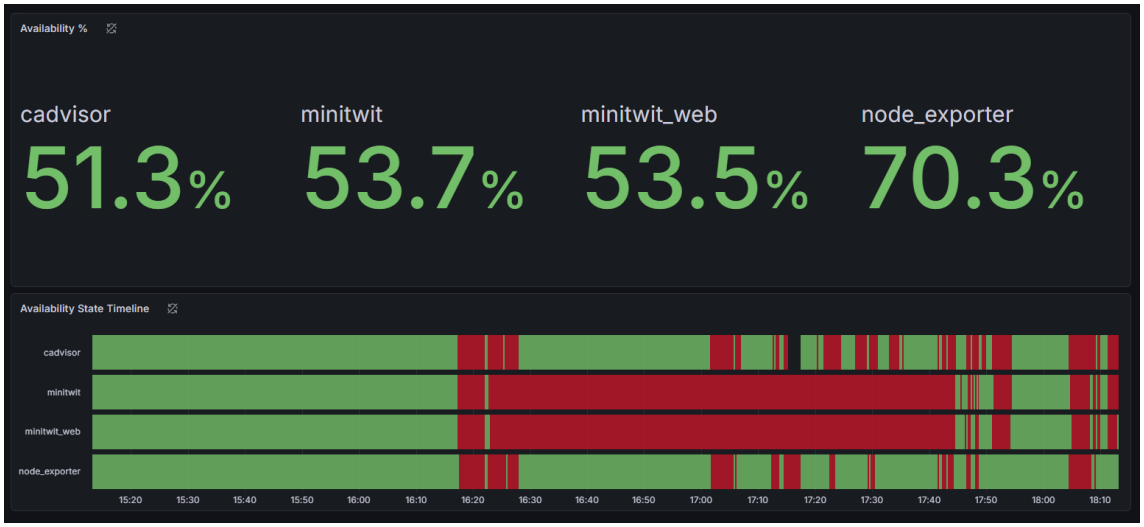


Figure 8: Dashboard of Availability

Resource Usage Dashboard

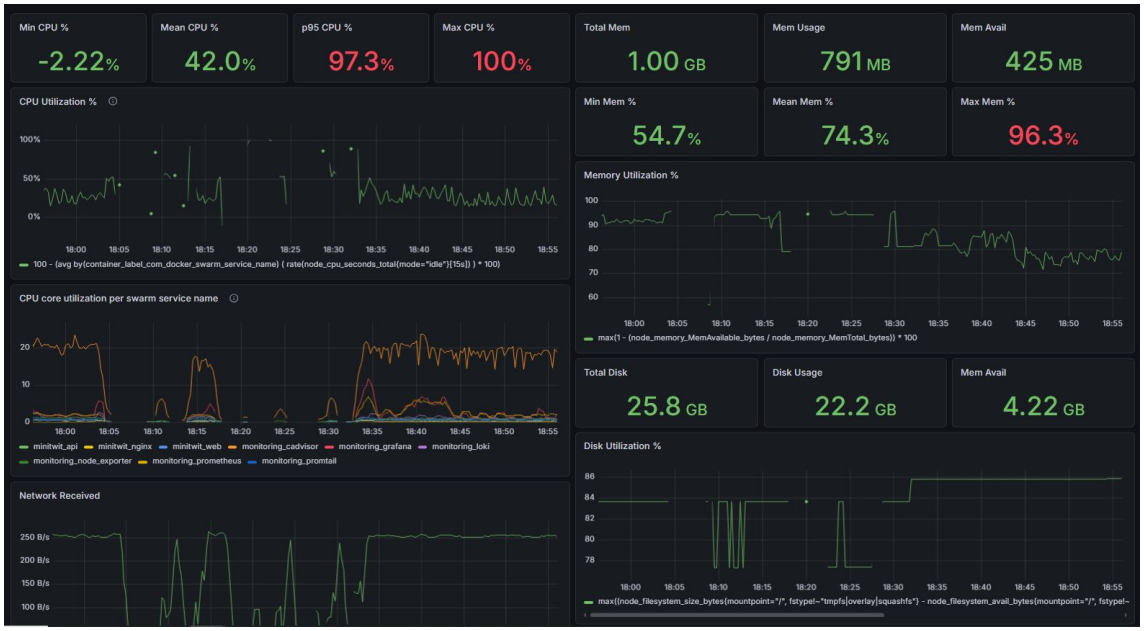


Figure 9: Dashboard of Resource Usage

5.2 Logging dashboard

Logging Dashboard

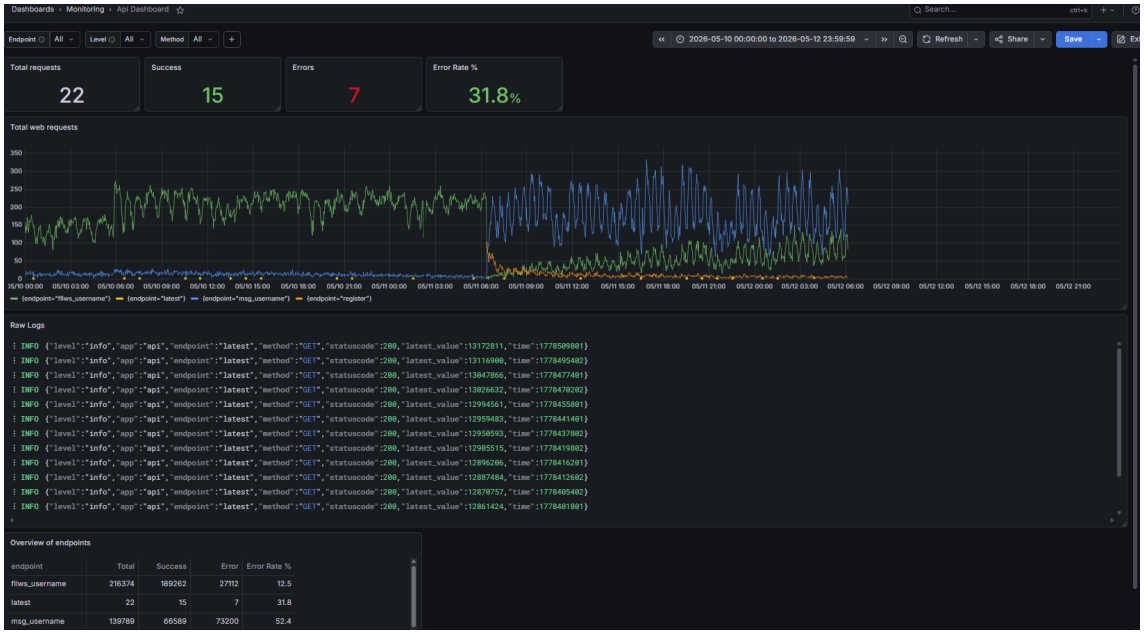


Figure 10: Dashboard for logging